从 " 算得出 " 到 " 算得对 "

LLM 运筹建模的语义校验、多链仲裁与自进化 harness

RetailOpt-190 × 内网 Qwen3.8-27B · 实验全记录 (19 个实验 , 逐数字可追溯)

2026-11-03 · github.com/Goldfish908/ora(round2 分支)· 文档 docs/paper/ · 原始记录 runs/

报告导览

- 背景与调研：自然语言 → 运筹优化建模 的任务设定，现有四类方法的结构性盲区
- Motivation 实验 (4 个): 假成功普遍存在、誊抄错误、先验覆盖翻转对、注入探针
- 系统：三层校验 harness —— 预求解闸门 / SCIP 求解 / witness 语义重判 / 修复环
- 工程主线 (8 个实验):memory 三代递进 → crash guard → 合体臂 → 三链集成仲裁 80.5%
- 对照与预算 :ReLoop 同模型严格对照、预算重分配、负结果 (如实报告)
- 自进化与科研命题：三层切片隔离、预注册生死判据、创新点逐条裁决

任务设定：一句话生成可执行的优化模型

- 数据集 RetailOpt-190:ReLoop 团队公开基准 (非自建);38 个原型 × 5 个数值变体 ,8 个问题族 F1–F8
  - ReLoop 论文报道该基准 SOTA 为 Claude Opus 4.6 的 31.1%
- 模型 : 内网 Qwen3.8-27B,thinking 开启 ,temperature 0.6; 求解器 SCIP / PySCIPOpt(全开源栈)
- **主指标 · 真实成功 = 代码可执行 ∧ witness 判语义可行 ∧ 目标值与参照解一致 (容差 1e-2)**
- 幻影 (spurious success): 代码跑通、目标值看似一致，但 witness 重判语义违例 —— 看似对实则错



调研：四类现有方法与结构性盲区

方法	核心思路	关键局限（我们的突破口）
ReLoop(2025)	执行反馈驱动的诊断 - 修复 - 回滚环	只看求解行为 ;feasible-but-wrong 是其自认的 silent failure
SAC-Opt	建模回译 + 自动语义对齐	不执行求解；发现错误 ≠ 能修复，尤难修结构性错误
Opt-Verifier(2026)	双侧验证 : 结构回译 + 解的 LLM 判读	LLM 判官 false-clear 率校准研究达 90%; 数据丰富实例用 toy 绕过
OptArgus(2026)	幻觉检测与分类 (31.1% 幻觉率)	只检测不修复；无执行级语义审计
本工作	**确定性 witness + 预求解闸门 + 多链仲裁 + prompt 进化**	**witness 规则按族定制，迁移性靠自进化线解决**

- 两轮文献扫描 + 逐条新颖性裁决见 docs/paper/related_work_scan_v0.1.md 与 novelty_audit_v0.1.md
- **结论 :" 确定性、零 LLM 、误报实测可控的语义审计 " 与 judge 路线互补，且在其不可靠处接管**

核心问题：幻影 —— 执行信号看不见的假成功

- **实测 (RetailOpt-190 全量 ,thinking 开):69 题出解 , 其中 46 题是幻影 , 真成功仅 13 题**
  - 目标值比对协议会报 32.1%(61/190), 真实仅 6.8% —— 高估 4.7 倍 ;off 臂更极端 : 报 23% 、实际 0%
- 幻影不是随机失误 : 全部幻影共享同一机制 (幻影初始库存), 目标值系统性虚低 7%–45%
- 根因 : 题面欠规约 (未声明 " 期初库存为零 "), 模型系统性占据规约留下的自由度
  - 即 specification gaming under underspecification —— 是被诱导的系统性行为 , 不是噪声
- **推论 : 任何只看执行信号的框架 (ReLoop 式修复环、结果奖励 RL) 结构性看不见这类错误**
- **→ 必须有一个与求解行为无关的独立语义审计器 (witness), 这是全工作的基石**

Motivation E1: 先剥离 " 誊抄错误 " 这一测量噪声

- 动机 : 测裸模型底力 , 定位大头失败模式 (4 个 F1 实例 × thinking 开 / 关)
- **发现 : 数据内嵌题面 (prompt_full) 下 4/4 全部死于数据誊抄 —— 把内嵌 JSON 抄成非法 Python**
- 对策 :prompt_schema 模式 —— 数据预加载为变量 , 物理上不可能誊抄 ; 崩溃率 7/8 → 3/8
- **测量纪律 : 此后全部实验固定 prompt_schema —— 先把测量噪声清掉 , 才谈语义错误**
- 启示 :LLM 建模的失败是分层的 (誊抄层 / 代码层 / 语义层), 测量与修复都必须分层

Motivation E2:witness 重判主表 —— 假成功普遍存在

模式	代码可执行	witness 可行	目标值一致	幻影	真实成功
thinking 关	48 / 190	0	0	43	**0(0%)**
thinking 开	69 / 190	15	15	46	**13(6.8%)**

- 做法 : 事后用 witness 对全部 190 题的产物重判 (与生成时的执行信号无关)
- **thinking 关 :43 个 " 看似成功 " 全部是幻影 , 真实成功为 0 —— 执行指标完全失真**
- 幻影机制高度一致 : 全部含 " 幻影初始库存 "(规则 C-INIT)—— 见下一页案例走查
- **结论 : 独立语义审计不是锦上添花 , 是让测量本身成立的必要仪器**

案例走查：一个幻影是如何产生并被抓到的

- 题目：零售库存，多保质期桶 (fresh → aging 逐期老化)，题面未声明期初库存
- **模型犯的错：给第 1 期的非新鲜桶凭空填库存 I[p,l,1,r>0] > 0 —— 免费现货替代采购**
- 后果：求解器愉快地给出更低成本（虚低 7%–45%）；代码跑通、目标值 " 对得上 "，执行信号一切正常
- **witness 的 C-INIT 规则：逐 (p,l,r) 核对 I[p,l,1,r]=0，违例即报 → 幻影被抓**
- 关键：这一步不需要标准答案，只需要题面语义 (" 库存从哪来 " 必须守恒)—— 因此可上线、可审计

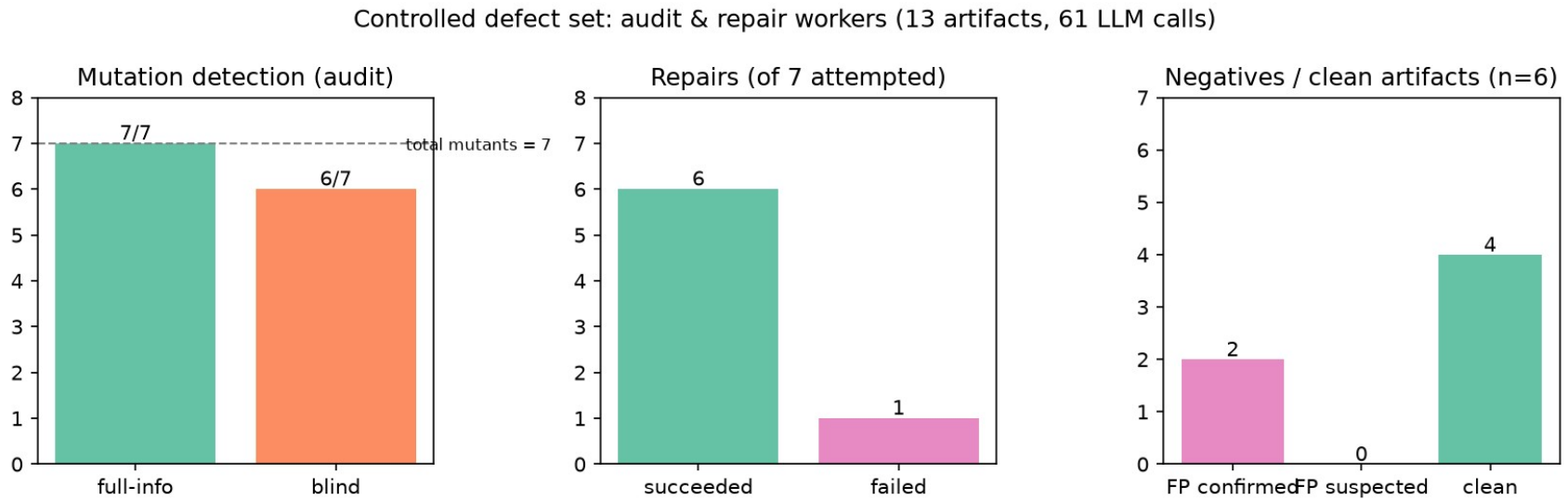
```
                          # m1_witness/retailopt.py —— witness 语义审计（零 LLM 的确定性检查）
#  机制由实例数据自动激活，而非按题面名称硬编码：
#    constraints.moq > 0     -> C-MOQ: Q ∈ {0} ∪ [moq, ∞)
#    costs.fixed_order > 0   -> C-FIXED 计入目标；lead_time > 0 -> C-INFLOW ...
for p in P:                           # C-INIT: 期初（第 1 期）非新鲜桶库存必须为 0
    for r in range(1, sl):
        iv = _get(I, p, l, 1, r)
        if abs(iv) > TOL:
            _violations_append(v, "C-INIT", f"I[{p},{l},1,{r}]={iv} != 0", cap)
```

Motivation E3: 翻转对 —— " 先验覆盖 " 真实存在

- 科研命题 : 模块之间的 " 连接 " 语义 (如库龄、提前期) 会被模型固有先验覆盖吗 ?thinking 会加剧吗 ?
- 方法 : 自建受控集翻转对 —— 同一结构两种语义 , 正确答案必须跟随题面翻转 ; 扩量到每格 48 样本
- **结果 ① 先验覆盖真实存在 , 且被先验强度调制 : 几乎只发生在 storage_time(关 35% / 开 21%), 其余连接 ≤ 6%**
- **结果 ② "thinking 加剧先验覆盖 " 不成立 :on 的题面跟随率全面 ≥ off**
- 结果 ③ 早期小样本的反向信号 (v4) 扩量后消失 —— 确认是噪声
- **意义 : 现象保住、错误归因拿掉 —— 预注册 + 扩量纪律防止了一个错误结论进入论文**

Motivation E4: 注入探针 —— 检查器的检出下限

- 做法：往正确建模里注入人为错误（变量 / 约束 / 符号类），看检查器能否发现
- **合成错误 100% 可检出 —— 下限达标；但真实错误显著更隐蔽，且检出依赖 thinking**
- 受控缺陷集（13 个 artifact、61 次 LLM 调用）：盲审检出 6/7 变异，修复成功 6/7，干净件误报 2/6
- **结论：注入探针只证明下限；真实分布才是战场 —— 主线全部使用真实 agent 轨迹做评估**



受控缺陷集：审计 worker 盲审（full-info 对照）与修复 worker 表现

系统总览：三层校验 harness（全程确定性、零额外 LLM）



- **设计原则：所有校验层都是确定性 Python，不调 LLM —— 误报率可实测、可复现、可审计**
- 闸门 fail-open：拿不准就放行，对成功样本通过率 84.6%，幻影误报 1.9%、成功样本误报 0%
- witness 代表出题人意图的语义一侧，与求解行为无关 —— 这是它与 ReLoop 式执行反馈的本质区别
- 并行扩展：三条多样化生成链独立跑，最后确定性仲裁（见 E12，80.5%）

witness 核心代码：机制从实例数据自动激活

```
                    """Generalized RetailOpt witness checker (all 190 instances, 38 archetypes).
Mechanisms activate from the instance data (not from the scenario name):
  costs.fixed_order > 0          -> C-FIXED in objective (per (p,l,t) with Q>0)
  constraints.moq > 0            -> C-MOQ: Q in {0} u [moq, inf)
  constraints.pack_size > 1      -> C-PACK: Q multiple of pack_size
  constraints.budget_per_period  -> C-BUDGET: per-period spend cap
  constraints.waste_limit_pct    -> C-WASTELIMIT: sum W <= pct * sum demand
  network.trans_edges non-empty  -> C-XEDGE/C-XNET, transshipment cost in obj
  labor_usage any > 0 ...        -> C-LABOR: usage * handled <= cap
  lead_time[p] > 0 / return_rate[p] > 0 -> folded into C-INFLOW
"""

for t in range(1, T + 1):
    inflow = fresh_inflow(p, l, t)          # C-INFLOW:  入库恒等式
    lhs = _get(I, p, l, t, sl)
    if abs(lhs - inflow) > TOL * max(1.0, abs(inflow)):
        _violations_append(v, "C-INFLOW", f"I[{p},{l},{t},{sl}]={lhs} != {inflow}")
```

- 18 条规则按族定制（当前局限：跨数据集迁移需重建，由自进化线的 checker-authoring 解决）

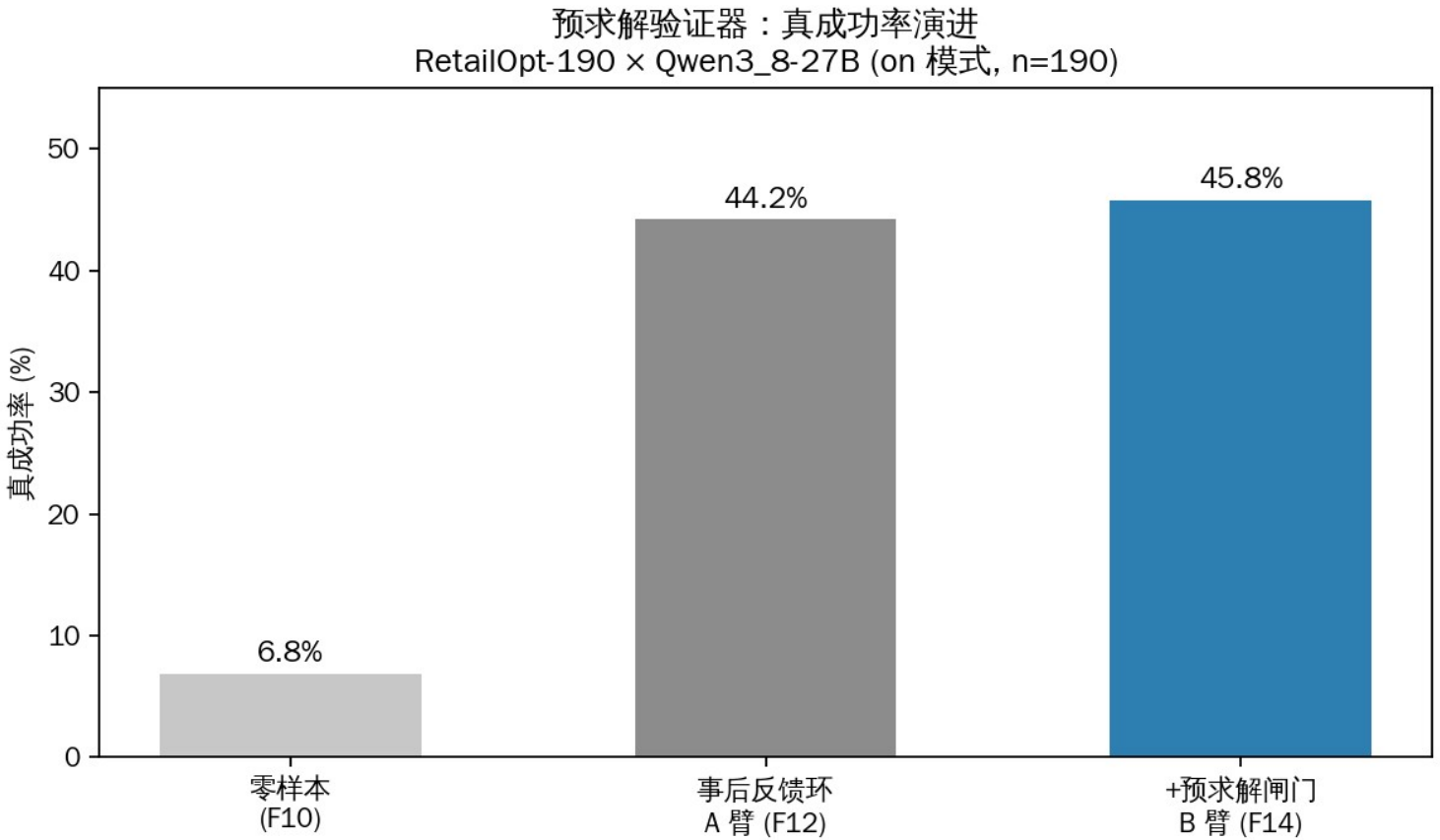
预求解闸门：求解之前拦截 " 注定判死 " 的代码

```
    """Deterministic AST probes over generated modeling code (Phase A).
Extracts, from a gen.py source string:
- data_keys: dataset keys the code actually reads
- var_names: solver variable name prefixes from addVar(name=...) calls
- init_signals: heuristic evidence that first-period inventory is pinned

Each probe maps a witness rule family to required data keys.
A probe FAILS (=> early BLOCK evidence) when a required key is never
referenced anywhere in the source.                                  """
```

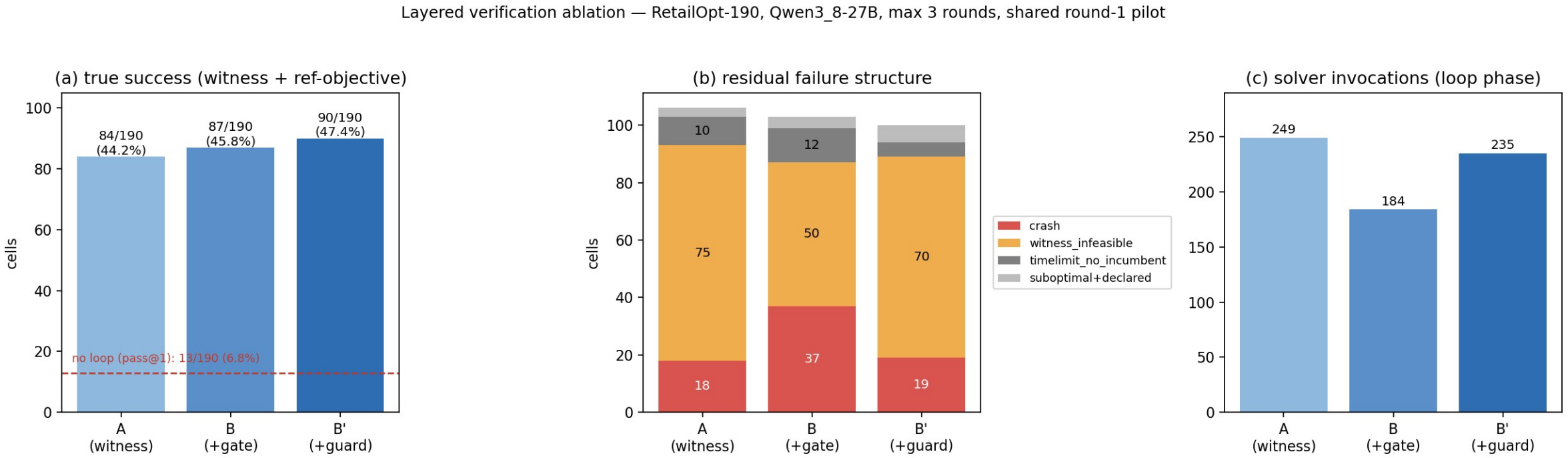
- **动机：省掉 " 注定被 witness 判死 " 的无效求解预算**
- 规则来自 263 条历史崩溃台账蒸馏，纯 Python，零 LLM
- **离线检出率迭代：73.8% → 91.6% → 93.9% → 97.0%(255/263)**
- 误报实测：幻影 1.9% 、成功样本 0%(fail-open)
- **在线对照 (190 题): 真成功 44.2% → 45.8%; 求解调用 – 26.1%, 墙钟 – 16.3%**
- 残余结构 :witness 不可行 75→50(−33%), 崩溃 18→37 —— 瓶颈从语义层转移到代码健壮性层 (设计预期)

闸门在线对照 :6.8% → 44.2% → 45.8%



F14: 零样本 / 事后反馈环 (A 臂)/ + 预求解闸门 (B 臂),RetailOpt-190 × Qwen3.8-27B(on 模式)

分层消融 :witness 环 → + 闸门 → +crash guard



A(仅 witness 环)44.2% → B(+ 闸门)45.8% → B'(+guard)47.4%; 残余失败结构随层迁移 ; 闸门省 26% 求解调用

memory 三代递进 :P1 → P2 → P3

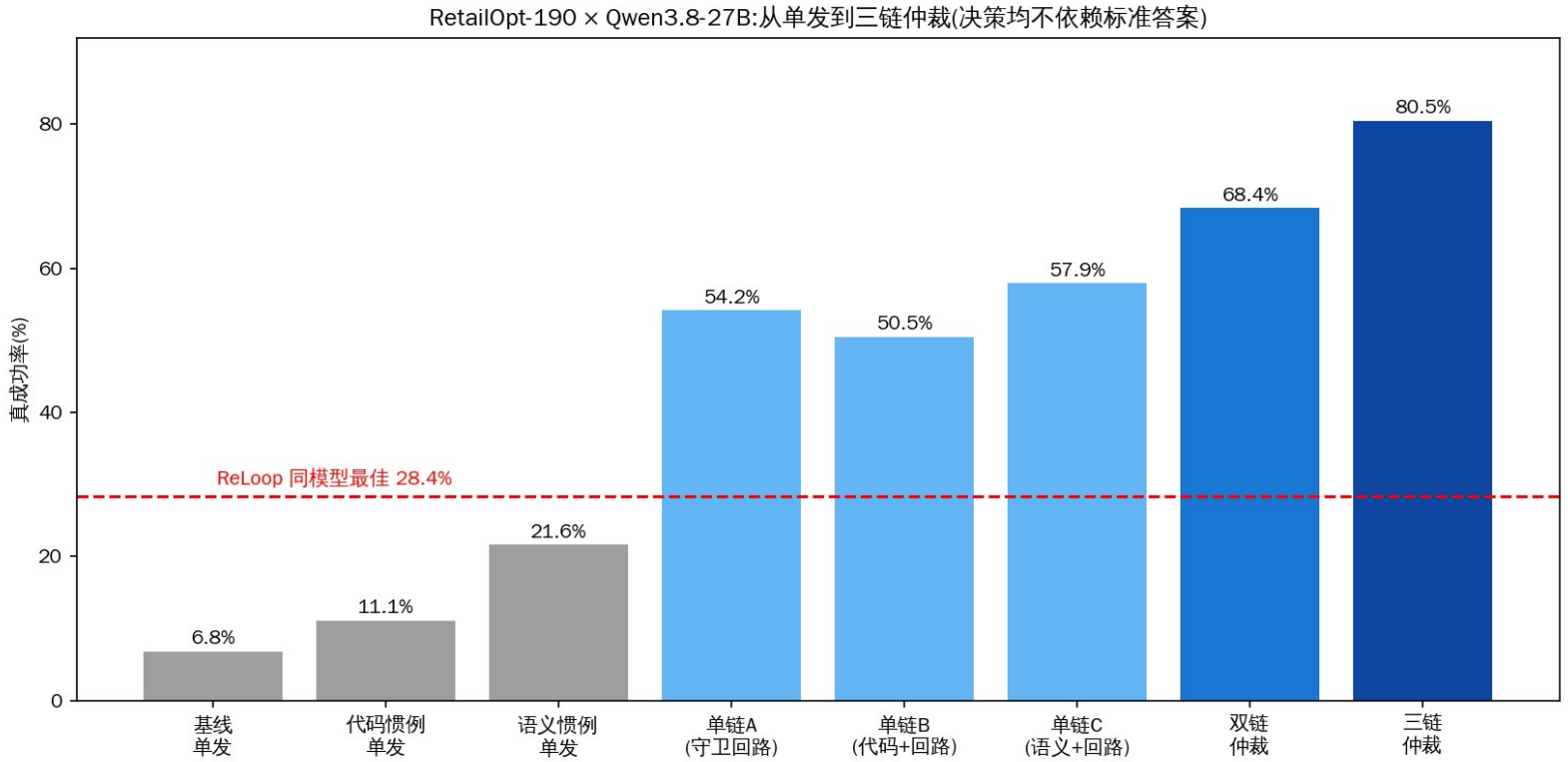
阶段	机制	单发 pass@1	关键观察
裸模型 (thinking 开)	—	6.8%	基线
P1 先验注入	G0–G8 条款 (防崩溃)	8.4%	崩溃 64%→7%, 但错误转化为幻影 (出解题中 90% 幻影)
P2 生成端升级	+ 建模程序脚手架	11.1%	对齐 ReLoop 生成端 , 成本几乎不变
P3 语义 memory	**+7 条语义惯例 , 族留出**	**21.6%**	**近乎翻倍 ; 幻影 – 12.6pp;F5 族 5%→40%; 全环 57.9%**

- **缺陷洗白 (创新点 D):P1 只修崩溃不修语义 , 把可见失败洗成不可见幻影 —— 修复越深 , 测量越不能只看执行信号**
- P3 的 7 条语义惯例 (需求恒等、库存平衡、初始库存置零、紧 Big-M 等) 蒸馏自反馈轨迹 , 冻结后注入 ; 留出族不参与蒸馏

Crash Guard 在线：环内实时拦截，崩溃残余再减半

- 动机：把崩溃规则做成环内实时拦截器 (guard)，修复环里即时命中即时改
- **B' 臂 (guard v0.1 + 闸门):47.4%; 崩溃残余 37 → 19(−49%);guard 在 43.5% 的细胞中触发**
- **B'' 臂 (guard v0.3 + 闸门):54.2%(103/190), 配对净翻转 +13; 逐轮累计 34.7% → 54.2%**
- 合体臂 (P2 生成端 + guard v0.4 + 闸门 + witness 环):r3 终态 50.5% —— 第三轮被 B'' 反超
- **关键发现 :B'' 与合体臂成功集交集仅 67 题、并集 132 题 (69.5%)—— 两条链高度互补**
- → 直接催生集成仲裁实验 (下一页)

三链集成仲裁 :80.5%, 距 Oracle 上界 1.1pp



三条单链 (54.2 / 50.5 / 57.9%)→ 双链 68.4% → 三链仲裁 80.5%; 虚线为 ReLoop 同模型最佳 28.4%

```
# scripts/ensemble_arb.py —— 确定性仲裁
# 决策时刻零真值、零 LLM:
cand = [r for r in all_rounds
        if r["witness"] == "feasible"]
# 最小化问题：可行候选中取目标值最小
best = min(cand, key=lambda c: c.obj)

# 并集上界 (Oracle, 需真值 )= 81.6%
# 仲裁规则 80.5%, 差距仅 1.1pp
```

- 部署视角 :80.5% vs 单链 57.9% 的差值 , 通常远大于 3× 求解成本

预算重分配与负结果（如实报告）

- **E13 预算重分配（纯离线重放，零新 LLM 调用）：**
  - 假设"砍掉修复环第 3 轮"不成立 —— 链 A 第 3 轮边际修复率 29.8% 与第 2 轮 29.9% 持平
  - 结论：预算优化方向是"先铺满多样性（链数），再加深轮数"，以及用自进化抬高单链起点
- **E14 witness 修复反馈增强消融 —— 干净的负结果：**
  - enriched 反馈（附规则明细）20.3% vs plain 22.0%，McNemar 检验 9v8，无显著差异
  - 决策：主线保持 plain 反馈，不并入增强；负结果同样落盘、推送、进实验清单
- E15 旁证 —— 成功预测器校准（53 个影子对）：预测概率与实际成功率单调一致，用于预算与调度决策

外部对照 :ReLoop 完整管线移植到同一模型

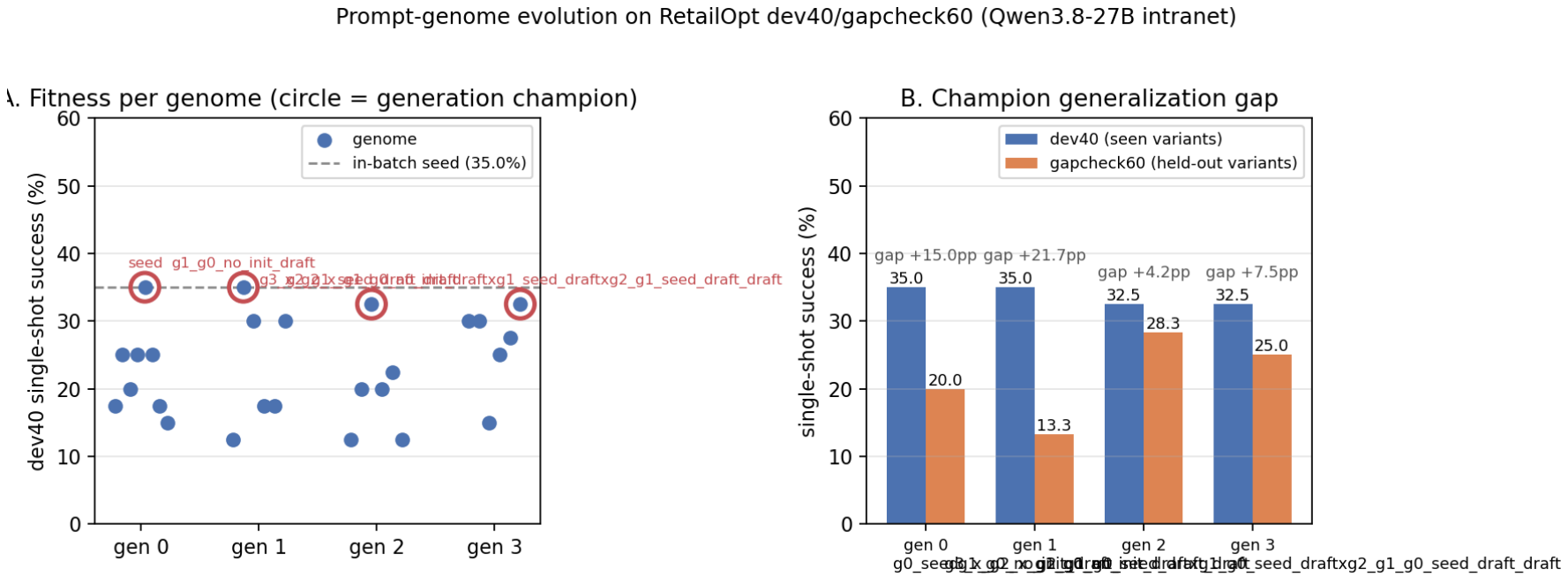
口径	ReLoop(同模型移植)	本工作
单发	R1t(CoT 单次)20.5%	P3 单发 21.6%
≤3 轮修复环	R2t(CoT + L1 环)28.4%	**P3 臂全环 57.9%**
多链集成	—	**三链仲裁 80.5%**

- **严格性 : 同模型 (Qwen3.8-27B) 、同数据集 (RetailOpt-190) 、同求解器 (SCIP);ReLoop 从其公开仓库完整移植**
- 附带发现 :thinking 是准确率的决定性变量 —— 关 3.7% vs 开 28.4%(×7.7)
- 公平性注记 :ReLoop 的环含诊断 - 回滚 ; 我们的比较按轮次对齐口径 , 全部数字留台账可追溯

自进化最小可行版：隔离协议 + 预注册生死判据

- 进化对象 :prompt 基因组 (memory 条款文本); 选择压力 :dev40 单发适应度
- **三层切片物理隔离 (sha256 落盘):**
  – dev40(F1–F4 的 v0+v4 变体 , 进化可见)/ gapcheck60(v1–v3 变体 , 过拟合闸门)/ F5–F8 留出族 (90 题 , 只用于最终评测)
- 每代 6 个后代 :2 个 LLM 条款草稿 (从精英失败记录起草)+ 1 语义块改写 + 2 交叉 + 1 最长条款删除
- **预注册生死判据 (结果出来前写死):**
  – (a) 第 3 代冠军 dev40 > 45.0%(同批种子 35.0% + 10pp)→ 显著性门槛
  – (c) 冠军全量 190 中 F5–F8 留出族 ≥ 15.0%(P3 的 20.0% − 5pp)→ 留出族不衰减
  – (b) dev40 vs gapcheck60 落差仅作诊断量 ;(a)∧(c)→ 继续 ,¬(a)→ 回退手工 + 集成 ,(a)∧¬(c)→ 判过拟合回退

进化收官 :gen0–gen3 全部落账，判据（a）不成立 → 回退



左 : 各基因组 dev40 适应度 (圈 = 该代冠军); 右 : 冠军的 dev40 vs gapcheck60 泛化落差

- **gen0: 种子 35.0% 夺冠 ; 删任一条款全掉点 → 条款内容本身有贡献**
- 噪声标定 : 同配置两次测量差 10pp(1σ≈7.5pp)→ 判据参照系用同批种子
- **落差 gen0 +15.0pp → gen1 +21.7pp(恶化)→ gen2 +4.2pp → gen3 +7.5pp**
  - 选择压力一度加剧 dev 过拟合 (gen1),gen2/3 回落但非 dev40 突破
- **判据 (a) 结果 :gen3 冠军 32.5% < 45% 门槛 —— 不成立**
  - 三代冠军 35.0→35.0→32.5→32.5%,LLM 变异无噪声外增益
- **按预注册规则回退 : 手工 P3 + 三链仲裁 (80.5%) 为部署主线**
  - 冠军全量 190:22.1% vs P3 21.6%(噪声带内无差异);F5–F8 留出族 15.6% ≥ 15% 阈值贴边通过

创新点逐条裁决（两轮文献扫描后）

候选贡献	裁决	一句话依据
A 幻影现象普遍存在	仅动机	OptArgus 31.1% 、 Falsification 27.6% 、 ReLoop 已命名 silent failure
B 机制归因 (specification gaming)	偏弱	通用层面已有；我们是 OR 域首个逐实例机制量化
C witness 确定性语义审计	拥挤 · 有差异化	vs Opt-Verifier 的 LLM 判官 :false-clear 90%; 我们误报 1.9%/0% 实测可控
D 缺陷洗白的纵向量化	**空位 ★**	**修复把可见失败洗成幻影；自然轨迹上未见度量者**
E 选择压力放大不可见缺陷	**空位 ★**	**进化落差 15→21.7pp 是域内首个实证**
F 双层留出 + 预注册判据	空位	成套测量纪律，与 D/E 咬合成故事闭环
G 多样性仲裁 80.5%(距 Oracle 1.1pp)	工程亮点	best-of-n + verifier 在通用层面拥挤
H/I 连接泛化 + 反例监督训练	空位 · 主战场	预注册 RQ1/RQ2, 科研线下一阶段

数字总览与下一步

配置	口径	真实成功
裸模型 (thinking 开)	单发	6.8%
P3 语义 memory	单发	**21.6%**
ReLoop 同模型 R1t / R2t	单发 / ≤3 轮	20.5% / 28.4%
B''(guard v0.3 + 闸门)	全环 ≤ 3 轮	54.2%
P3 臂	全环 ≤ 3 轮	57.9%
三链仲裁	**离线重放**	**80.5%(Oracle 81.6%)**

- **权属 :RetailOpt-190 为 ReLoop 团队公开基准 ; 全部对比同模型同数据集同求解器**
- **下一步 :**

– 自进化判据已裁决 : 回退手工 P3 + 三链仲裁 (见进化结果页); 教训沉淀为论文证据 (选择压力放大不可见缺陷)

– checker-authoring agent: 让 witness 规则自动蒸馏 , 解决跨数据集迁移

– IndustryOR 零适配迁移预实验 (检验非参数进化的可迁移性)

– 反例监督训练 (预注册 RQ2): 把 witness 判出的执行反例灌进训练

- 全部材料 :docs/paper/(24 份专项文档)+ runs/ 原始台账 , 逐数字可追溯